THE WARRIOR'S

DOCUMENTATION ARSENAL

Complete Gear Checklist

Databricks · Apache Spark · Django

192	1022	8	11
TOTAL GEAR	TOTAL XP	WEAPONS	SECTIONS

WARRIOR RANKS

Rank	Title	Unlock condition
I	Raw Recruit	0 items — the forge awaits
II	Squire	10 items equipped
III	Scout	25 items equipped
IV	Soldier	45 items equipped
V	Knight	65 items equipped
VI	Warlord	85 items equipped
VII	Champion	100 items equipped
VIII	**DATA LEGEND**	**All 192 items — full arsenal unlocked**

[ ] checkbox	Tick when you have read and practised the topic
+Nxp	Experience points earned — track your power level
Gear name	What this chapter equips you with in warrior terms
URL	Exact documentation page to open on your device

TABLE OF CONTENTS — THE FULL ARSENAL

- [ ] Auto Loader: rescued data column
- [ ] Auto Loader vs COPY INTO — when to use each
- [ ] COPY INTO — idempotent file ingestion

[FLAME BANNER] Delta Live Tables (DLT)

7 items · 38 xp

- [ ] What is Delta Live Tables?
- [ ] DLT: streaming vs materialised view tables
- [ ] DLT expectations (WARN / DROP / FAIL)
- [ ] DLT pipeline: development vs production mode
- [ ] DLT: triggered vs continuous pipelines
- [ ] Monitoring DLT with event logs
- [ ] DLT with Unity Catalog

[BOOTS] Spark SQL — Mobility & Speed

23 items · 134 xp

- [ ] SparkSession — the entry point
- [ ] SELECT, FROM, WHERE, LIMIT
- [ ] GROUP BY, HAVING, ORDER BY
- [ ] INNER JOIN, LEFT JOIN, RIGHT JOIN
- [ ] FULL OUTER JOIN, CROSS JOIN
- [ ] Semi-join and anti-join
- [ ] UNION, INTERSECT, EXCEPT (set operations)
- [ ] Subqueries and correlated subqueries
- [ ] CTEs (WITH clause)
- [ ] Window functions: RANK, DENSE_RANK, ROW_NUMBER
- [ ] Window functions: LAG, LEAD, NTILE
- [ ] Window functions: SUM/AVG OVER partitions
- [ ] PIVOT and UNPIVOT
- [ ] CASE WHEN / IF / IIF
- [ ] String functions (regexp_replace, split, trim)
- [ ] Date and time functions
- [ ] EXPLODE, FLATTEN — array functions
- [ ] TRANSFORM, FILTER, AGGREGATE — HOF
- [ ] Struct and Map type operations
- [ ] Parsing JSON with from_json / to_json
- [ ] NULL handling: COALESCE, NULLIF, IS NULL
- [ ] SQL UDFs: CREATE FUNCTION
- [ ] Broadcast hints and query hints

[GAUNTLETS] PySpark DataFrame API — Striking Power

26 items · 138 xp

- [ ] DataFrame creation: spark.read.csv/json/parquet
- [ ] spark.createDataFrame from Python list/pandas
- [ ] Schema definition with StructType/StructField
- [ ] select() and selectExpr()
- [ ] filter() / where()
- [ ] withColumn() and withColumnRenamed()
- [ ] drop() and dropDuplicates()
- [ ] groupBy() and agg()
- [ ] join() with all join types
- [ ] orderBy() / sort()
- [ ] limit() and sample()
- [ ] union() and unionByName()
- [ ] na.fill(), na.drop(), na.replace()
- [ ] Actions: show(), count(), collect(), first()
- [ ] write.format().mode().save()
- [ ] createOrReplaceTempView() / createGlobalTempView()
- [ ] Method chaining patterns
- [ ] Python UDFs: udf() decorator and registration
- [ ] Pandas UDFs (vectorised UDFs) with @pandas_udf

[ ] explain() — reading query plans
[ ] cache() and persist()
[ ] Broadcast variables and accumulators
[ ] Spark MLlib Pipeline API
[ ] MLlib: transformers and estimators
[ ] MLlib: classification, regression models
[ ] MLlib: feature engineering transformers

[HELMET] Databricks Workflows — Command & Strategy

15 items · 72 xp

[ ] Jobs overview — what is a Databricks job?
[ ] Creating a job with notebook task
[ ] Job task types: notebook, Python, JAR, SQL, DLT
[ ] Multi-task workflows and dependencies
[ ] Passing parameters between tasks (task values)
[ ] Cron scheduling syntax
[ ] File-arrival triggers
[ ] Concurrent run policies (allow, skip, wait)
[ ] Retry policies for failed tasks
[ ] Email alerts on success/failure
[ ] Viewing job run history and logs
[ ] Databricks CLI setup and authentication
[ ] CLI: jobs run now, clusters list
[ ] REST API: trigger jobs programmatically
[ ] Secrets management for pipeline credentials

[CLOAK] Unity Catalog — Stealth & Governance

13 items · 60 xp

[ ] Unity Catalog overview and benefits
[ ] Three-level namespace: catalog.schema.table
[ ] Metastore vs legacy Hive metastore (HMS)
[ ] CREATE CATALOG, CREATE SCHEMA
[ ] GRANT and REVOKE on catalogs, schemas, tables
[ ] Privilege types: SELECT, MODIFY, CREATE, USE
[ ] Service principals vs user accounts
[ ] Data owner and data steward roles
[ ] Automatic column-level data lineage
[ ] Audit logs for data access
[ ] Row filters using dynamic views
[ ] Column masks for PII data
[ ] Tags and data discovery

[MEDALLION] Medallion Architecture — The Warrior's Sacred Code

6 items · 36 xp

[ ] Medallion architecture overview
[ ] Bronze layer — raw ingestion design
[ ] Silver layer — validation and cleansing
[ ] Gold layer — business aggregates
[ ] Connecting layers with DLT pipelines
[ ] Data quality enforcement per layer

[SPELL TOME — Book I] Django — Models & Database (ORM Spells)

18 items · 100 xp

[ ] Django overview — MTV architecture
[ ] Starting a project and app structure
[ ] Models — defining fields and field types
[ ] Model relationships: ForeignKey, ManyToMany, OneToOne
[ ] Meta class — ordering, verbose_name, indexes
[ ] Migrations: makemigrations and migrate
[ ] Custom migrations and data migrations
[ ] QuerySet API: filter, exclude, get, all
[ ] Chaining QuerySets and lazy evaluation

- [ ] annotate() and aggregate() — F and Q objects
- [ ] select_related() — forward FK traversal
- [ ] prefetch_related() — reverse/M2M traversal
- [ ] values(), values_list() — flat output
- [ ] order_by(), distinct(), count(), exists()
- [ ] Raw SQL with raw() and connection.execute()
- [ ] Custom model managers
- [ ] Model signals (post_save, pre_delete)
- [ ] Database transactions and atomic()

[SPELL TOME — Book II] Django — Views, URLs & HTTP (Battle Dispatch)

15 items · 71 xp

- [ ] Function-based views (FBV)
- [ ] Class-based views (CBV)
- [ ] Generic CBVs: ListView, DetailView, CreateView
- [ ] URL configuration and path()/re_path()
- [ ] URL namespaces and reverse()
- [ ] Middleware — request/response processing
- [ ] HttpRequest and HttpResponse objects
- [ ] Django forms and form validation
- [ ] Sessions and cookies
- [ ] Caching framework
- [ ] Authentication: login, logout, permissions
- [ ] Custom user model and AbstractUser
- [ ] Django admin — auto-generated management UI
- [ ] Static files and media files serving
- [ ] Testing in Django — TestCase, Client

[SPELL TOME — Book III] Django REST Framework — API Forge (ML Delivery)

17 items · 100 xp

- [ ] DRF overview and installation
- [ ] Serializers — ModelSerializer and Serializer
- [ ] SerializerMethodField and custom validation
- [ ] APIView — the base class
- [ ] ViewSets and Routers — automatic URL generation
- [ ] Generic views: ListCreateAPIView, RetrieveUpdateDestroyAPIView
- [ ] Authentication: Token, Session, JWT
- [ ] Permissions: IsAuthenticated, IsAdminUser, custom
- [ ] Throttling and rate limiting
- [ ] Filtering, searching, ordering query params
- [ ] Pagination: PageNumberPagination, CursorPagination
- [ ] Nested serializers and writable relations
- [ ] Serving a scikit-learn model via DRF endpoint
- [ ] API versioning strategies
- [ ] Django deployment: Gunicorn + Nginx
- [ ] Environment variables and settings management
- [ ] Dockerising a Django + ML application

[SHIELD] Databricks — Lakehouse Platform

Your body armour. Every concept here protects your data architecture from collapse.

Source: Databricks Data Engineering Docs | docs.databricks.com/aws/en/data-engineering/

- [ ] **What is the Databricks Lakehouse Platform?** +5xp
  Iron Breastplate — the foundational concept of the entire exam
  docs.databricks.com/aws/en/introduction/index.html
- [ ] **Lakehouse vs data warehouse vs data lake** +5xp
  Chest plate — know the battlefield before you fight on it
  docs.databricks.com/aws/en/lakehouse/index.html
- [ ] **Databricks architecture: control plane & data plane** +4xp
  Spine guard — understand who controls what
  docs.databricks.com/aws/en/getting-started/overview.html
- [ ] **Databricks Runtime versions** +3xp
  Armour alloy — choose the right material for the job
  docs.databricks.com/aws/en/release-notes/runtime/index.html
- [ ] **All-purpose clusters** +4xp
  Standard shield — general combat cluster
  docs.databricks.com/aws/en/compute/configure.html
- [ ] **Job clusters** +4xp
  Battle shield — lightweight, auto-terminates after battle
  docs.databricks.com/aws/en/compute/configure.html
- [ ] **Cluster modes: single node, standard, high concurrency** +4xp
  Shield variants — pick your defence posture
  docs.databricks.com/aws/en/compute/cluster-config-best-practices.html
- [ ] **Auto-scaling and auto-termination** +4xp
  Shield auto-repair — never waste gold on idle armour
  docs.databricks.com/aws/en/compute/configure.html
- [ ] **Cluster policies** +3xp
  Shield governance — enforce rules across the army
  docs.databricks.com/aws/en/compute/cluster-policies.html
- [ ] **Databricks notebooks — cell types & magic commands** +5xp
  Shield emblem — %sql %python %md %run mastery
  docs.databricks.com/aws/en/notebooks/index.html
- [ ] **Notebook widgets** +3xp
  Shield handle — parameterise your notebooks
  docs.databricks.com/aws/en/notebooks/widgets.html
- [ ] **Notebook-scoped libraries** +3xp
  Shield polish — install libraries per notebook
  docs.databricks.com/aws/en/libraries/notebook-libraries.html
- [ ] **Databricks Repos — Git integration** +4xp
  Shield lineage — version control your battle plans
  docs.databricks.com/aws/en/repos/index.html
- [ ] **Branching, committing, pulling in Repos** +4xp
  Shield reinforcement — collaborate without overwriting
  docs.databricks.com/aws/en/repos/git-operations-with-repos.html
- [ ] **Databricks SQL & SQL warehouses overview** +4xp
  Shield signal — query data for business commanders
  docs.databricks.com/aws/en/sql/index.html
- [ ] **Serverless vs classic SQL warehouses** +3xp
  Shield tier selection — cost vs speed tradeoff
  docs.databricks.com/aws/en/compute/sql-warehouse/create.html
- [ ] **Connecting BI tools to Databricks SQL** +3xp
  Shield API port — plug Power BI and Tableau in
  docs.databricks.com/aws/en/integrations/bi/index.html

SECTION COMPLETE · 17 items · 65 xp earned

[SWORD] Delta Lake — Your Primary Weapon

Your main offensive weapon. Master every strike here — Delta is 22% of the DEA exam.

Source: Databricks Delta Lake Docs | docs.databricks.com/aws/en/delta/

- [ ] **What is Delta Lake?** +6xp
  Sword forged — ACID on top of Parquet, the foundation
  docs.databricks.com/aws/en/delta/index.html
- [ ] **The _delta_log transaction log** +6xp
  Sword spine — where every commit is recorded
  docs.databricks.com/aws/en/delta/transaction-log.html
- [ ] **Managed vs external Delta tables** +5xp
  Blade type A vs B — know which you control
  docs.databricks.com/aws/en/delta/managed-vs-unmanaged.html
- [ ] **CREATE TABLE USING DELTA** +5xp
  First forge strike — create your weapon
  docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-create-table-using.html
- [ ] **DESCRIBE DETAIL and DESCRIBE HISTORY** +5xp
  Blade inspection — examine your weapon status
  docs.databricks.com/aws/en/delta/history.html
- [ ] **ALTER TABLE — schema changes** +5xp
  Blade reshaping — add columns, rename, evolve schema
  docs.databricks.com/aws/en/delta/column-mapping.html
- [ ] **Schema enforcement & schema evolution** +6xp
  Blade hardening — control what data enters
  docs.databricks.com/aws/en/delta/schema-evolution.html
- [ ] **MERGE INTO — upserts (MOST TESTED)** +10xp
  DIAMOND EDGE — the ultimate strike: insert+update+delete
  docs.databricks.com/aws/en/sql/language-manual/delta-merge-into.html
- [ ] **UPDATE and DELETE on Delta tables** +6xp
  Surgical strikes — precision modifications
  docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-dml-update.html
- [ ] **Slowly Changing Dimensions with MERGE (SCD1/2)** +7xp
  Advanced combo — maintain historical records
  docs.databricks.com/aws/en/delta/merge.html
- [ ] **Time travel: VERSION AS OF** +8xp
  Temporal strike — attack with historical data
  docs.databricks.com/aws/en/delta/history.html
- [ ] **Time travel: TIMESTAMP AS OF** +7xp
  Time anchor — pinpoint a moment in history
  docs.databricks.com/aws/en/delta/history.html
- [ ] **RESTORE TABLE to previous version** +6xp
  Blade restoration — undo battle damage
  docs.databricks.com/aws/en/sql/language-manual/delta-restore.html
- [ ] **VACUUM — cleaning old files** +5xp
  Blade cleaning kit — remove obsolete file versions
  docs.databricks.com/aws/en/sql/language-manual/delta-vacuum.html
- [ ] **OPTIMIZE — file compaction** +6xp
  Blade honing — compact small files into powerful strikes
  docs.databricks.com/aws/en/sql/language-manual/delta-optimize.html
- [ ] **Z-ORDER BY — multi-dimensional clustering** +6xp
  Blade focus — co-locate related data for faster reads
  docs.databricks.com/aws/en/delta/data-skipping.html
- [ ] **Data skipping with column statistics** +5xp
  Feint technique — skip irrelevant data entirely
  docs.databricks.com/aws/en/delta/data-skipping.html

- [ ] **ANALYZE TABLE** +4xp
  Battle intelligence — update column statistics
  `docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-dml-analyze-table.html`
- [ ] **Table properties and options** +4xp
  Blade enchantments — configure table behaviours
  `docs.databricks.com/aws/en/delta/table-properties.html`
- [ ] **Clone Delta tables (SHALLOW and DEEP)** +4xp
  Mirror blade — duplicate without duplicating cost
  `docs.databricks.com/aws/en/delta/clone.html`

SECTION COMPLETE · 20 items · 116 xp earned

[SPEAR] Structured Streaming & Auto Loader

Your ranged weapon. Strike data before it lands. Real-time is your long-range attack.

`Source: Databricks Structured Streaming Docs | docs.databricks.com/aws/en/structured-streaming/`

- [ ] **Structured Streaming overview** +6xp
  Spear shaft forged — the foundation of real-time combat
  `docs.databricks.com/aws/en/structured-streaming/index.html`
- [ ] **Streaming sources: Delta, Kafka, files** +5xp
  Spear targets — where your stream comes from
  `docs.databricks.com/aws/en/structured-streaming/data-sources.html`
- [ ] **Streaming sinks: Delta, memory, console** +5xp
  Spear landing zones — where your stream goes
  `docs.databricks.com/aws/en/structured-streaming/data-sources.html`
- [ ] **Output modes: append, complete, update** +7xp
  Strike modes — how data lands at the target
  `docs.databricks.com/aws/en/structured-streaming/output-modes.html`
- [ ] **Trigger modes: once, continuous, availableNow** +7xp
  Strike speed — control your throw frequency
  `docs.databricks.com/aws/en/structured-streaming/triggers.html`
- [ ] **Checkpointing and fault tolerance** +7xp
  Spear lanyard — never lose your throw, always recover
  `docs.databricks.com/aws/en/structured-streaming/query-recovery.html`
- [ ] **Watermarking for late data** +6xp
  Spear range limiter — handle late-arriving data
  `docs.databricks.com/aws/en/structured-streaming/watermarks.html`
- [ ] **Reading Delta as a streaming source** +6xp
  Infinite quiver — Delta feeds the stream endlessly
  `docs.databricks.com/aws/en/structured-streaming/delta-streaming.html`
- [ ] **Writing streams to Delta sinks** +6xp
  Strike landing — persist stream results to Delta
  `docs.databricks.com/aws/en/structured-streaming/delta-streaming.html`
- [ ] **foreachBatch for custom logic** +5xp
  Custom spear tip — apply any transformation mid-flight
  `docs.databricks.com/aws/en/structured-streaming/foreach.html`
- [ ] **Auto Loader overview (cloudFiles)** +8xp
  Auto-loading crossbow — monitor cloud storage automatically
  `docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/index.html`
- [ ] **Auto Loader: schema inference and hints** +6xp
  Arrow shape detection — automatically detect incoming data shape
  `docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema.html`
- [ ] **Auto Loader: rescued data column** +5xp
  Broken arrow recovery — capture malformed records safely
  `docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/rescue.html`

- [ ] **Auto Loader vs COPY INTO — when to use each** +7xp
  Weapon selection — crossbow vs cannon, know the difference
  `docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/index.html`
- [ ] **COPY INTO — idempotent file ingestion** +6xp
  Siege cannon — load files once, safely, idempotently
  `docs.databricks.com/aws/en/sql/language-manual/delta-copy-into.html`

SECTION COMPLETE · 15 items · 92 xp earned

[FLAME BANNER] Delta Live Tables (DLT)

Your battle standard. DLT pipelines self-heal, enforce quality, and inspire the whole army.

`Source: Databricks Delta Live Tables Docs | docs.databricks.com/aws/en/delta-live-tables/`

- [ ] **What is Delta Live Tables?** +6xp
  Banner unfurled — declarative, self-healing pipelines
  `docs.databricks.com/aws/en/delta-live-tables/index.html`
- [ ] **DLT: streaming vs materialised view tables** +6xp
  Banner variants — live view vs saved snapshot
  `docs.databricks.com/aws/en/delta-live-tables/tables.html`
- [ ] **DLT expectations (WARN / DROP / FAIL)** +7xp
  Banner quality laws — enforce or punish bad data
  `docs.databricks.com/aws/en/delta-live-tables/expectations.html`
- [ ] **DLT pipeline: development vs production mode** +5xp
  Training ground vs battlefield — switch modes deliberately
  `docs.databricks.com/aws/en/delta-live-tables/updates.html`
- [ ] **DLT: triggered vs continuous pipelines** +5xp
  Burst charge vs sustained assault — pick your battle tempo
  `docs.databricks.com/aws/en/delta-live-tables/updates.html`
- [ ] **Monitoring DLT with event logs** +5xp
  Battle debrief — read the pipeline event log
  `docs.databricks.com/aws/en/delta-live-tables/observability.html`
- [ ] **DLT with Unity Catalog** +4xp
  Banner + cloak — governed, lineage-tracked pipelines
  `docs.databricks.com/aws/en/delta-live-tables/unity-catalog.html`

SECTION COMPLETE · 7 items · 38 xp earned

[BOOTS] Spark SQL — Mobility & Speed

Speed across any terrain. Spark SQL lets you move through distributed data at full scale.

`Source: Apache Spark SQL Reference + PySpark SQL Docs | spark.apache.org/docs/latest/api/python/reference/pyspark.sql/`

- [ ] **SparkSession — the entry point** +5xp
  Boots buckled — spark.sql() and spark.read()
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/spark_session.html`
- [ ] **SELECT, FROM, WHERE, LIMIT** +4xp
  Basic step — every journey starts here
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **GROUP BY, HAVING, ORDER BY** +5xp
  Formation march — aggregate and sort your forces
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **INNER JOIN, LEFT JOIN, RIGHT JOIN** +6xp
  Force merging — combine armies from two fronts
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`

- [ ] **FULL OUTER JOIN, CROSS JOIN** +5xp
  Total war — every possible combination of forces
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **Semi-join and anti-join** +6xp
  Spy tactics — filter by what exists elsewhere
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **UNION, INTERSECT, EXCEPT (set operations)** +5xp
  Army merging laws — combine or subtract forces
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **Subqueries and correlated subqueries** +6xp
  Tactical manoeuvre — query within a query
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **CTEs (WITH clause)** +6xp
  Agility rune — name your sub-battles cleanly
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **Window functions: RANK, DENSE_RANK, ROW_NUMBER** +8xp
  Speed enchantment I — rank your soldiers
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html`
- [ ] **Window functions: LAG, LEAD, NTILE** +8xp
  Speed enchantment II — look forward and back
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html`
- [ ] **Window functions: SUM/AVG OVER partitions** +7xp
  Speed enchantment III — running totals across groups
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/window.html`
- [ ] **PIVOT and UNPIVOT** +6xp
  Formation flip — rotate rows to columns and back
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **CASE WHEN / IF / IIF** +5xp
  Conditional strike — branch your battle logic
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/conditional.html`
- [ ] **String functions (regexp_replace, split, trim)** +5xp
  Blade precision — cut and reshape text data
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/string.html`
- [ ] **Date and time functions** +5xp
  Temporal mastery — parse, format, and diff dates
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/datetime.html`
- [ ] **EXPLODE, FLATTEN — array functions** +7xp
  Parkour boots — expand nested collections into rows
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html`
- [ ] **TRANSFORM, FILTER, AGGREGATE — HOF** +7xp
  Combo parkour — apply logic inside arrays in-place
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html`
- [ ] **Struct and Map type operations** +6xp
  Terrain navigation — work with complex nested types
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/collection.html`
- [ ] **Parsing JSON with from_json / to_json** +6xp
  Cipher boots — decode and encode JSON payloads
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/misc.html`
- [ ] **NULL handling: COALESCE, NULLIF, IS NULL** +5xp
  Gap-filling soles — handle missing terrain safely
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/conditional.html`
- [ ] **SQL UDFs: CREATE FUNCTION** +6xp
  Custom boot mould — define your own SQL functions
  `docs.databricks.com/aws/en/udf/index.html`
- [ ] **Broadcast hints and query hints** +5xp
  Boots afterburner — guide the optimizer manually
  `spark.apache.org/docs/latest/sql-performance-tuning.html`

SECTION COMPLETE · 23 items · 134 xp earned

[GAUNTLETS] PySpark DataFrame API — Striking Power

Your bare-hand combat power. Code you write yourself — no AI crutch in interviews.

Source: PySpark DataFrame API Reference | spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html

- [ ] **DataFrame creation: spark.read.csv/json/parquet** +5xp
  Gauntlet base — pick up raw data with your hands
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **spark.createDataFrame from Python list/pandas** +4xp
  Forge data from nothing — create test datasets
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/spark_session.html
- [ ] **Schema definition with StructType/StructField** +5xp
  Gauntlet shaping — define exact data structure
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/types.html
- [ ] **select() and selectExpr()** +5xp
  First strike — pick the columns you need
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **filter() / where()** +5xp
  Defensive parry — keep only what you want
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **withColumn() and withColumnRenamed()** +5xp
  Knuckle reshape — add and rename columns
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **drop() and dropDuplicates()** +4xp
  Clean strike — remove columns and duplicate rows
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **groupBy() and agg()** +6xp
  Power grip — group and aggregate with force
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **join() with all join types** +6xp
  Two-handed slam — combine two DataFrames
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **orderBy() / sort()** +4xp
  Formation order — rank your results
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **limit() and sample()** +3xp
  Controlled swing — cap or sample your data
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **union() and unionByName()** +4xp
  Force combination — stack two DataFrames
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **na.fill(), na.drop(), na.replace()** +5xp
  Wound treatment — handle nulls in your data
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **Actions: show(), count(), collect(), first()** +5xp
  Strike delivery — trigger actual computation
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **write.format().mode().save()** +5xp
  Store your spoils — write results to storage
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **createOrReplaceTempView() / createGlobalTempView()** +5xp
  Bridge building — switch between API and SQL
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html
- [ ] **Method chaining patterns** +5xp
  Combo strikes — chain 5 transformations smoothly
  spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html

- [ ] **Python UDFs: udf() decorator and registration** +7xp
  Gauntlet spike I — add custom Python logic
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/udf.html`
- [ ] **Pandas UDFs (vectorised UDFs) with @pandas_udf** +7xp
  Gauntlet spike II — 10x faster custom functions
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions/udf.html`
- [ ] **explain() — reading query plans** +6xp
  X-ray vision — see how Spark executes your code
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html`
- [ ] **cache() and persist()** +5xp
  Gauntlet charge — hold data in memory for reuse
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/dataframe.html`
- [ ] **Broadcast variables and accumulators** +5xp
  War supplies — share data across the cluster
  `spark.apache.org/docs/latest/api/python/reference/pyspark.sql/index.html`
- [ ] **Spark MLlib Pipeline API** +7xp
  Magic gauntlet — chain ML stages like transformations
  `spark.apache.org/docs/latest/api/python/reference/pyspark.ml/index.html`
- [ ] **MLlib: transformers and estimators** +6xp
  Spell components — the two types of ML pipeline stages
  `spark.apache.org/docs/latest/api/python/reference/pyspark.ml/index.html`
- [ ] **MLlib: classification, regression models** +7xp
  Combat spells — RandomForest, LogisticRegression at scale
  `spark.apache.org/docs/latest/api/python/reference/pyspark.ml/classification.html`
- [ ] **MLlib: feature engineering transformers** +7xp
  Spell ingredients — StringIndexer, VectorAssembler, Scaler
  `spark.apache.org/docs/latest/api/python/reference/pyspark.ml/feature.html`

SECTION COMPLETE · 26 items · 138 xp earned

[HELMET] Databricks Workflows — Command & Strategy

Your strategic mind protection. Orchestrate armies of tasks without losing a single soldier.

`Source: Databricks Workflows & Jobs Docs | docs.databricks.com/aws/en/jobs/`

- [ ] **Jobs overview — what is a Databricks job?** +4xp
  Helmet base — the command structure explained
  `docs.databricks.com/aws/en/jobs/index.html`
- [ ] **Creating a job with notebook task** +4xp
  First command — dispatch a notebook to battle
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Job task types: notebook, Python, JAR, SQL, DLT** +5xp
  Helmet variants — command different types of troops
  `docs.databricks.com/aws/en/jobs/index.html`
- [ ] **Multi-task workflows and dependencies** +6xp
  Tactical chain of command — sequential and parallel tasks
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Passing parameters between tasks (task values)** +6xp
  Command signals — share intelligence across tasks
  `docs.databricks.com/aws/en/jobs/share-task-context.html`
- [ ] **Cron scheduling syntax** +5xp
  Battle clock — trigger attacks on exact schedule
  `docs.databricks.com/aws/en/jobs/schedule-jobs.html`
- [ ] **File-arrival triggers** +5xp
  Sentry trigger — auto-launch when data arrives
  `docs.databricks.com/aws/en/jobs/schedule-jobs.html`

- [ ] **Concurrent run policies (allow, skip, wait)** +4xp
  Queue discipline — manage overlapping battles
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Retry policies for failed tasks** +5xp
  Battle revival — configure automatic retries
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Email alerts on success/failure** +4xp
  War messenger — notify commanders of outcomes
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Viewing job run history and logs** +4xp
  After-action report — examine every past battle
  `docs.databricks.com/aws/en/jobs/create-run-jobs.html`
- [ ] **Databricks CLI setup and authentication** +5xp
  Radio headset — command the army remotely
  `docs.databricks.com/aws/en/dev-tools/cli/index.html`
- [ ] **CLI: jobs run now, clusters list** +5xp
  Voice commands — trigger and inspect via terminal
  `docs.databricks.com/aws/en/dev-tools/cli/index.html`
- [ ] **REST API: trigger jobs programmatically** +5xp
  Remote command centre — automate from any system
  `docs.databricks.com/aws/en/dev-tools/api/index.html`
- [ ] **Secrets management for pipeline credentials** +5xp
  Cipher helmet — store passwords safely, never in code
  `docs.databricks.com/aws/en/security/secrets/index.html`

SECTION COMPLETE · 15 items · 72 xp earned

[CLOAK] Unity Catalog — Stealth & Governance

Controls who sees what. The cloak of invisibility and access control over your data kingdom.

`Source: Databricks Unity Catalog Docs | docs.databricks.com/aws/en/data-governance/unity-catalog/`

- [ ] **Unity Catalog overview and benefits** +4xp
  Cloak woven — centralised governance across workspaces
  `docs.databricks.com/aws/en/data-governance/unity-catalog/index.html`
- [ ] **Three-level namespace: catalog.schema.table** +6xp
  Cloak layers — master the full hierarchy
  `docs.databricks.com/aws/en/data-governance/unity-catalog/index.html`
- [ ] **Metastore vs legacy Hive metastore (HMS)** +4xp
  Old cloak vs new — know the upgrade path
  `docs.databricks.com/aws/en/data-governance/unity-catalog/index.html`
- [ ] **CREATE CATALOG, CREATE SCHEMA** +5xp
  Cloak territory — define your data kingdom zones
  `docs.databricks.com/aws/en/data-governance/unity-catalog/create-catalogs.html`
- [ ] **GRANT and REVOKE on catalogs, schemas, tables** +6xp
  Cloak seal — decide who enters which zone
  `docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/index.html`
- [ ] **Privilege types: SELECT, MODIFY, CREATE, USE** +5xp
  Access levels — different keys for different doors
  `docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges.html`
- [ ] **Service principals vs user accounts** +4xp
  Spy vs soldier — machine identity vs human identity
  `docs.databricks.com/aws/en/administration-guide/users-groups/index.html`
- [ ] **Data owner and data steward roles** +4xp
  Kingdom roles — know the chain of authority
  `docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/index.html`

- [ ] **Automatic column-level data lineage** +5xp
  Cloak tracker — see every data footprint
  `docs.databricks.com/aws/en/data-governance/unity-catalog/data-lineage.html`
- [ ] **Audit logs for data access** +4xp
  Spy journal — log every door that was opened
  `docs.databricks.com/aws/en/administration-guide/account-settings/audit-logs.html`
- [ ] **Row filters using dynamic views** +5xp
  Cloak patch I — hide specific rows from enemies
  `docs.databricks.com/aws/en/data-governance/unity-catalog/row-and-column-filters.html`
- [ ] **Column masks for PII data** +5xp
  Cloak patch II — mask sensitive columns automatically
  `docs.databricks.com/aws/en/data-governance/unity-catalog/row-and-column-filters.html`
- [ ] **Tags and data discovery** +3xp
  Kingdom map — label and find your data assets
  `docs.databricks.com/aws/en/data-governance/unity-catalog/tags.html`

SECTION COMPLETE · 13 items · 60 xp earned

[MEDALLION] Medallion Architecture — The Warrior's Sacred Code

Bronze, Silver, Gold — the three sacred laws every clean pipeline must obey.

`Source: Databricks Medallion Architecture Guide | docs.databricks.com/aws/en/lakehouse/medallion.html`

- [ ] **Medallion architecture overview** +5xp
  Sacred code revealed — the three-layer philosophy
  `docs.databricks.com/aws/en/lakehouse/medallion.html`
- [ ] **Bronze layer — raw ingestion design** +6xp
  Bronze medallion earned — store everything raw, lose nothing
  `docs.databricks.com/aws/en/lakehouse/medallion.html`
- [ ] **Silver layer — validation and cleansing** +7xp
  Silver medallion earned — purify, deduplicate, enforce quality
  `docs.databricks.com/aws/en/lakehouse/medallion.html`
- [ ] **Gold layer — business aggregates** +7xp
  Gold medallion earned — serve only what decisions need
  `docs.databricks.com/aws/en/lakehouse/medallion.html`
- [ ] **Connecting layers with DLT pipelines** +6xp
  Sacred chain — wire Bronze to Silver to Gold with DLT
  `docs.databricks.com/aws/en/lakehouse/medallion.html`
- [ ] **Data quality enforcement per layer** +5xp
  Quality vows — what each layer must guarantee
  `docs.databricks.com/aws/en/lakehouse/medallion.html`

SECTION COMPLETE · 6 items · 36 xp earned

[SPELL TOME — Book I] Django — Models & Database (ORM Spells)

Your magic spellbook Part I. Command databases with Python incantations.

`Source: Django Documentation – Models & Database Layer | docs.djangoproject.com/en/5.2/topics/db/`

- [ ] **Django overview — MTV architecture** +4xp
  Tome cover — Model, Template, View: the three pillars
  `docs.djangoproject.com/en/5.2/intro/overview/`
- [ ] **Starting a project and app structure** +4xp
  Spell preparation — manage.py, settings, apps
  `docs.djangoproject.com/en/5.2/intro/tutorial01/`

- [ ] **Models — defining fields and field types** +6xp
  Spell page 1 — CharField, IntegerField, ForeignKey and more
  `docs.djangoproject.com/en/5.2/topics/db/models/`
- [ ] **Model relationships: ForeignKey, ManyToMany, OneToOne** +6xp
  Relational runes — link your data kingdoms
  `docs.djangoproject.com/en/5.2/topics/db/models/#relationships`
- [ ] **Meta class — ordering, verbose_name, indexes** +4xp
  Tome chapter meta — control model behaviour
  `docs.djangoproject.com/en/5.2/ref/models/options/`
- [ ] **Migrations: makemigrations and migrate** +6xp
  Spell evolution — upgrade your schema safely
  `docs.djangoproject.com/en/5.2/topics/migrations/`
- [ ] **Custom migrations and data migrations** +5xp
  Advanced spell casting — write your own schema spells
  `docs.djangoproject.com/en/5.2/topics/migrations/#data-migrations`
- [ ] **QuerySet API: filter, exclude, get, all** +7xp
  Summon spell — call forth exactly the data you need
  `docs.djangoproject.com/en/5.2/ref/models/querysets/`
- [ ] **Chaining QuerySets and lazy evaluation** +6xp
  Spell delay — QuerySets are lazy until you need them
  `docs.djangoproject.com/en/5.2/ref/models/querysets/#when-querysets-are-evaluated`
- [ ] **annotate() and aggregate() — F and Q objects** +7xp
  Calculation runes — compute new values on the fly
  `docs.djangoproject.com/en/5.2/topics/db/aggregation/`
- [ ] **select_related() — forward FK traversal** +7xp
  Efficiency rune I — fetch related records in one query
  `docs.djangoproject.com/en/5.2/ref/models/querysets/#select-related`
- [ ] **prefetch_related() — reverse/M2M traversal** +7xp
  Efficiency rune II — defeat the N+1 query curse forever
  `docs.djangoproject.com/en/5.2/ref/models/querysets/#prefetch-related`
- [ ] **values(), values_list() — flat output** +5xp
  Spell simplification — return dicts instead of objects
  `docs.djangoproject.com/en/5.2/ref/models/querysets/#values`
- [ ] **order_by(), distinct(), count(), exists()** +5xp
  Sorting and counting spells
  `docs.djangoproject.com/en/5.2/ref/models/querysets/`
- [ ] **Raw SQL with raw() and connection.execute()** +5xp
  Ancient tongue — drop to raw SQL when needed
  `docs.djangoproject.com/en/5.2/topics/db/sql/`
- [ ] **Custom model managers** +5xp
  Spell specialisation — add your own QuerySet methods
  `docs.djangoproject.com/en/5.2/topics/db/managers/`
- [ ] **Model signals (post_save, pre_delete)** +5xp
  Spell triggers — react automatically to data events
  `docs.djangoproject.com/en/5.2/topics/signals/`
- [ ] **Database transactions and atomic()** +6xp
  Spell isolation — wrap operations in ACID safety
  `docs.djangoproject.com/en/5.2/topics/db/transactions/`

SECTION COMPLETE · 18 items · 100 xp earned

[SPELL TOME — Book II] Django — Views, URLs & HTTP (Battle Dispatch)

Your magic spellbook Part II. Route requests to the right incantation.

`Source: Django Documentation — Views & HTTP Layer | docs.djangoproject.com/en/5.2/topics/http/`

- [ ] **Function-based views (FBV)** +5xp
  Simple spell cast — request in, response out
  `docs.djangoproject.com/en/5.2/topics/http/views/`
- [ ] **Class-based views (CBV)** +6xp
  Spell class hierarchy — reusable, extensible views
  `docs.djangoproject.com/en/5.2/topics/class-based-views/`
- [ ] **Generic CBVs: ListView, DetailView, CreateView** +5xp
  Spell shortcuts — pre-built views for common patterns
  `docs.djangoproject.com/en/5.2/ref/class-based-views/`
- [ ] **URL configuration and path()/re_path()** +5xp
  Spell routing table — map URLs to views cleanly
  `docs.djangoproject.com/en/5.2/topics/http/urls/`
- [ ] **URL namespaces and reverse()** +4xp
  Return path rune — never hardcode a URL again
  `docs.djangoproject.com/en/5.2/topics/http/urls/#naming-url-patterns`
- [ ] **Middleware — request/response processing** +5xp
  Spell interceptors — transform every request globally
  `docs.djangoproject.com/en/5.2/topics/http/middleware/`
- [ ] **HttpRequest and HttpResponse objects** +4xp
  Spell vessels — the carrier of all battle messages
  `docs.djangoproject.com/en/5.2/ref/request-response/`
- [ ] **Django forms and form validation** +5xp
  Input shield — validate data before it enters your realm
  `docs.djangoproject.com/en/5.2/topics/forms/`
- [ ] **Sessions and cookies** +4xp
  Spell memory — remember the warrior between visits
  `docs.djangoproject.com/en/5.2/topics/http/sessions/`
- [ ] **Caching framework** +5xp
  Spell memory cache — avoid repeating the same incantation
  `docs.djangoproject.com/en/5.2/topics/cache/`
- [ ] **Authentication: login, logout, permissions** +6xp
  Castle gates — control who enters your kingdom
  `docs.djangoproject.com/en/5.2/topics/auth/`
- [ ] **Custom user model and AbstractUser** +5xp
  Identity rune — define your own warrior identity
  `docs.djangoproject.com/en/5.2/topics/auth/customizing/`
- [ ] **Django admin — auto-generated management UI** +4xp
  Commander panel — manage your realm without code
  `docs.djangoproject.com/en/5.2/ref/contrib/admin/`
- [ ] **Static files and media files serving** +3xp
  Supply depot — serve images, CSS, JS in development
  `docs.djangoproject.com/en/5.2/howto/static-files/`
- [ ] **Testing in Django — TestCase, Client** +5xp
  War games — test your spells before battle
  `docs.djangoproject.com/en/5.2/topics/testing/`

SECTION COMPLETE · 15 items · 71 xp earned

[SPELL TOME — Book III] Django REST Framework — API Forge (ML Delivery)

Your most powerful tome. DRF lets you forge APIs that serve your ML models to the world.

`Source: Django REST Framework Documentation | django-rest-framework.org/`

- [ ] **DRF overview and installation** +5xp
  Tome III cover — the weapon for ML model delivery
  `django-rest-framework.org/`

- [ ] **Serializers — ModelSerializer and Serializer** +7xp
  Data translator — convert models to JSON and back
  `django-rest-framework.org/api-guide/serializers/`
- [ ] **SerializerMethodField and custom validation** +6xp
  Advanced translation — add computed fields and rules
  `django-rest-framework.org/api-guide/serializers/#serializermethodfield`
- [ ] **APIView — the base class** +6xp
  Spell base class — handle GET, POST, PUT, DELETE
  `django-rest-framework.org/api-guide/views/`
- [ ] **ViewSets and Routers — automatic URL generation** +6xp
  Auto-routing rune — generate all endpoints from one class
  `django-rest-framework.org/api-guide/viewsets/`
- [ ] **Generic views: ListCreateAPIView, RetrieveUpdateDestroyAPIView** +6xp
  Power shortcuts — CRUD in 5 lines
  `django-rest-framework.org/api-guide/generic-views/`
- [ ] **Authentication: Token, Session, JWT** +7xp
  API gate — control who calls your ML endpoints
  `django-rest-framework.org/api-guide/authentication/`
- [ ] **Permissions: IsAuthenticated, IsAdminUser, custom** +6xp
  Spell permission system — who can do what
  `django-rest-framework.org/api-guide/permissions/`
- [ ] **Throttling and rate limiting** +5xp
  Spell cooldown — protect your ML API from overload
  `django-rest-framework.org/api-guide/throttling/`
- [ ] **Filtering, searching, ordering query params** +5xp
  API refinement — let callers filter your ML outputs
  `django-rest-framework.org/api-guide/filtering/`
- [ ] **Pagination: PageNumberPagination, CursorPagination** +4xp
  Result management — deliver data in chunks
  `django-rest-framework.org/api-guide/pagination/`
- [ ] **Nested serializers and writable relations** +5xp
  Deep spell structure — represent related objects in one response
  `django-rest-framework.org/api-guide/relations/`
- [ ] **Serving a scikit-learn model via DRF endpoint** +10xp
  THE ULTIMATE SPELL — load model, run prediction, return JSON
  `django-rest-framework.org/tutorial/quickstart/`
- [ ] **API versioning strategies** +4xp
  Spell versioning — v1, v2 without breaking existing callers
  `django-rest-framework.org/api-guide/versioning/`
- [ ] **Django deployment: Gunicorn + Nginx** +6xp
  Release ritual — unleash your API in production
  `docs.djangoproject.com/en/5.2/howto/deployment/`
- [ ] **Environment variables and settings management** +5xp
  Spell secrets — never hardcode passwords in spells
  `docs.djangoproject.com/en/5.2/topics/settings/`
- [ ] **Dockerising a Django + ML application** +7xp
  Armour containerisation — ship your entire arsenal in a box
  `docs.djangoproject.com/en/5.2/howto/deployment/`

SECTION COMPLETE · 17 items · 100 xp earned